

ESP32 UNDERWATER SENSOR NODE Hardware Prototype

Stage 2 Technical Assessment

Pranav Duse

T.E. ECE

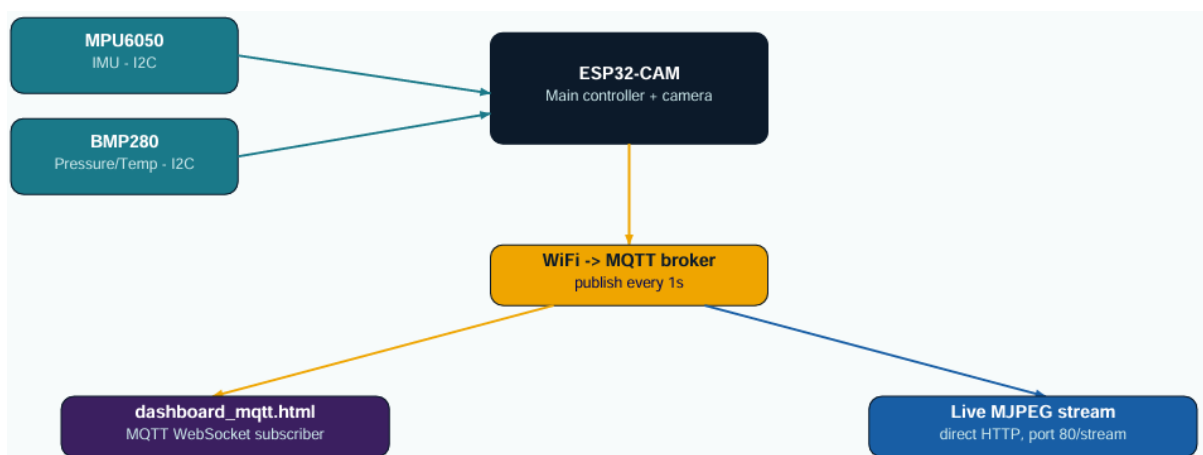
PES Modern College of Engineering, Pune

June 2026

Project Introduction

This project is a sensor node built using an ESP32-CAM board. It reads data from two sensors - an MPU6050 (which measures motion and tilt) and a BMP280 (which measures air pressure and temperature). The readings are sent over WiFi using MQTT to a dashboard webpage, where they show up live. The ESP32-CAM's built-in camera also streams live video to the same webpage, so both the sensor readings and the video can be seen together in one browser window. The goal of this stage was to build something that actually works end to end - sensors talking to the board, the board sending data over MQTT, and a dashboard that shows it live - rather than just a design on paper.

System Architecture



The ESP32-CAM reads both sensors every half second and sends the readings over MQTT once every second. MQTT is a simple way for small devices to send short messages to a server (called a broker) without needing a complicated setup. The dashboard webpage connects to the same broker and listens for these messages, updating the numbers on screen as soon as they arrive. The video works separately from the sensor data. The ESP32-CAM also runs as a small web server, and the camera feed is pulled directly into the dashboard page as a live video stream. This means the video and the sensor numbers are two separate connections running side by side - if one stops working, the other keeps going.

Components Used And Why

Part	What it does	Why it was chosen
ESP32-CAM	Main board + camera	Has WiFi, a camera, and enough pins for sensors - all in one board
MPU6050	Measures tilt and motion	Cheap, easy to find, works over I2C
BMP280	Measures pressure and temperature	Easy to get quickly; used here to test the full pipeline
MQTT broker	Passes messages between board and dashboard	Lightweight and simple to set up on a laptop
Dashboard webpage	Shows live data and video	Just a webpage - no extra app or software needed to view it

Note: The brief mentioned MS5837 as the preferred pressure/depth sensor, since it is rated for actual underwater use. That sensor was not available in time, so BMP280 was used instead to test and confirm that the full pipeline reading sensors, sending data, displaying it live - works correctly. Swapping in MS5837 later would not require changing this design, since it would connect the same way. There is also no SD card in this version. Data is only sent live over MQTT and is not saved locally on the board.

What the Dashboard Shows

- Live readings from the MPU6050 - tilt (roll and pitch), and raw motion values
- Live readings from the BMP280 - temperature, pressure, and estimated altitude
- Whether each sensor is working properly (a simple OK/Fault indicator)
- Live video from the camera, shown directly in the same page
- Basic connection status - whether MQTT is connected, and how often messages are arriving

Problems Faced and How They Were Solved

- MPU6050 would not initialize properly using the standard library function. This was fixed by manually sending the correct startup commands to the sensor over I2C (basically telling it directly to wake up and start measuring), instead of relying only on the library.
- BMP280 readings sometimes failed even though the sensor was detected. To fix this, the code was set up to fall back to reading the sensor's raw data directly if the normal method failed a few times in a row.
- Two BMP280-type sensors use different I2C addresses (0x76 or 0x77), and some cheaper modules are actually a different sensor (BME280) pretending to be a BMP280. The code checks the sensor's ID number directly to confirm which one is connected before using it.
- Video streaming and sensor readings competing for the board's attention - since the ESP32 has two processor cores, the video streaming was made to run on one core while sensor reading and MQTT sending run on the other, so neither slows down the other.
- No SD card or MS5837 sensor available in time - both were left out of this build and noted as something to add later, rather than working around them.

What Could Be Added Next

- Replace BMP280 with MS5837 for a sensor that is actually rated for underwater use
- Add an SD card so sensor data is also saved locally, not just sent live
- Use a proper online MQTT broker with a username/password instead of a local one
- Add a leak sensor and automatic shutdown, similar to the Stage 1 design
- Make the WiFi/MQTT reconnect more reliable so short network drops don't lose data

